

Etapa 2 - Analizador Sintáctico de MiniJava

Compiladores e Intérpretes — UNS, 2026

Implementación de un analizador sintáctico descendente recursivo para MiniJava, cliente del analizador léxico desarrollado en la Etapa 1. La gramática original de MiniJava no es LL(1), por lo que primero se la transformó (eliminación de recursión a izquierda y factorización simple y profunda) obteniendo una gramática equivalente LL(1), sobre la que se aplicó la estrategia recursiva simple: un método por cada no terminal, selección de la producción según los primeros de sus partes derechas y consumo de terminales mediante `match`.

1 Esquema general

El módulo principal recibe el archivo fuente y se lo asocia al manejador de archivos y este al léxico (como en la Etapa 1). Luego crea al sintáctico con la dependencia al léxico y le pide que comience el análisis. Si el análisis finaliza sin errores, informa el mensaje de éxito; si se produce un error sintáctico o léxico, el sintáctico termina propagando una excepción que el módulo principal captura y reporta. A diferencia de la Etapa 1, el principal no imprime la secuencia de tokens: su interfaz se limita a disparar el análisis y reportar el resultado.

2 Estructura del proyecto

```
codigo/
|-- Compilar.sh / Compilar.bat          Compila y genera Compilador.jar
|-- CorrerTests.sh / CorrerTests.bat    Compila y ejecuta los testers JUnit
|-- lib/                                Jars de JUnit 4 y Hamcrest (se autodescargan)
|-- src/main/java/
|   |-- moduloprincipal/
|   |   |-- ModuloPrincipal.java        Modulo principal (interfaz con el usuario)
|   |-- analizadorlexico/
|   |   |-- AnalizadorLexico.java       Automata finito determinista (Etapa 1, sin cambios)
|   |   |-- Token.java                 Representacion de un token (nombre, lexema, linea)
|   |   |-- ExcepcionLexica.java        Excepcion propagada ante un error lexico
|   |-- analizadorsintactico/
|   |   |-- AnalizadorSintactico.java    Parser descendente recursivo (un metodo por NT)
|   |   |-- ExcepcionSintactica.java     Excepcion propagada ante un error sintactico
|   |-- sourcemanager/
|   |   |-- SourceManager.java           Interfaz provista por la catedra
|   |   |-- SourceManagerImpl.java       Implementacion provista (por lineas, referencia)
|   |   |-- SourceManagerEficiente.java Implementacion propia por caracteres (se usa)
|-- src/test/java/test/                 Testers JUnit provistos por la catedra
|-- resources/
|   |-- sinErrores/                     Casos de prueba sin errores
|   |-- conErrores/                     Casos de prueba con errores
```

3 Componentes

3.1 Módulo principal (ModuloPrincipal)

Recibe por parámetro la ruta del archivo fuente MiniJava (acepta cualquier extensión):

```
java -jar Compilador.jar programa1.java
```

Asocia el archivo a un `SourceManagerEficiente`, crea el `AnalizadorLexico`, crea el `AnalizadorSintactico` con esa dependencia y le solicita el análisis. Al finalizar sin errores imprime el mensaje de éxito y `[SinErrores]`. Ante un error sintáctico o léxico captura la excepción correspondiente, reporta el mensaje y el código de error y finaliza la ejecución, como exigen las pautas de esta etapa.

3.2 Analizador sintáctico (`AnalizadorSintactico`)

Implementa la estrategia recursiva simple sobre la gramática LL(1) de la sección 5:

- **Un método por cada no terminal** de la gramática. El cuerpo codifica las partes derechas de sus producciones: por cada terminal llama a `match`, y por cada no terminal llama a su método.
- **Selección por primeros**: cuando un NT tiene varias producciones, se elige la primera cuya parte derecha tiene al token actual en su conjunto de primeros (conjuntos precalculados como constantes, p.ej. los primeros de un tipo o de una expresión). Si el NT tiene una producción ϵ , el esquema simple no hace nada en la rama `else`.
- **match**: compara el token actual con el terminal esperado; si coinciden pide el próximo token al léxico, y si no lanza `ExcepcionSintactica` con el lexema y la línea del token actual.
- **Arranque**: el método `analizar()` carga el primer token del léxico y llama al método del símbolo inicial `<Inicial>`.
- Un mapa de descripciones de tokens permite generar mensajes del estilo de las pautas (`se esperaba un id de clase se encontro ...`).

El sintáctico consume tokens bajo demanda: como el símbolo inicial exige llegar al EOF, todo token del fuente es requerido y, por lo tanto, un error léxico lanzado por el léxico se propaga a través del sintáctico hasta el módulo principal, que lo reporta con el mismo formato que un error sintáctico.

3.3 `ExcepcionSintactica`

Excepción (no verificada, al igual que `ExcepcionLexica`) con el lexema del token ofensor, su número de línea y un mensaje descriptivo de la naturaleza del error.

4 Transformación de la gramática a LL(1)

La gramática de MiniJava no es LL(1) por dos tipos de problemas: producciones con **recursión a izquierda** y producciones de un mismo NT cuyas partes derechas **comparten primeros** (no factorizadas). Se resolvieron en ese orden: primero la eliminación de recursión a izquierda y luego la factorización simple y profunda. Las técnicas aplicadas fueron las vistas en los temas 3.2.3 y 3.2.4. En lo que sigue se escribe `eps` por la producción vacía.

4.1 Eliminación de recursión a izquierda

- **Expresiones compuestas** (recursión por operador binario a la izquierda):

```
<ExpresionCompuesta> ::= <ExpresionCompuesta> <OperadorBinario> <ExpresionBasica>
                        | <ExpresionBasica>
```

se transforma en:

```
<ExpresionCompuesta> ::= <ExpresionBasica> <ECResto>
<ECResto> ::= <OperadorBinario> <ExpresionBasica> <ECResto> | eps
```

- **Referencias encadenadas** (recursión a izquierda por VarEncadenada, MetodoEncadenado y AccesoArreglo):

```
<Referencia> ::= <Referencia> <VarEncadenada>
                | <Referencia> <MetodoEncadenado>
                | <Referencia> <AccesoArreglo>
                | <Primario>
```

se transforma en:

```
<Referencia> ::= <Primario> <RResto>
<RResto> ::= . idMetVar <MetodoOVar> <RResto> | <AccesoArreglo> <RResto> | eps
```

- **Lista de argumentos formales:**

```
<ListaArgsFormales> ::= <ListaArgsFormales> , <ArgFormal> | <ArgFormal>
```

se transforma en:

```
<ListaArgsFormales> ::= <ArgFormal> <LAFResto>
<LAFResto> ::= , <ArgFormal> <LAFResto> | eps
```

4.2 Factorización simple y profunda

- **Miembros de una clase** (<Atributo>, <Metodo> y <Constructor> comparten el prefijo <Tipo>; además <Metodo> tenía un <ModificadorOpcional> que podía ser static). Factorización profunda:

```
<Miembro> ::= <Tipo> idMetVar <MetodoOAtributo> | <MetodoVoid>
                | static <MetodoResto> | <Constructor>
<MetodoOAtributo> ::= ; | <ArgsFormales> <Bloque>
<MetodoVoid> ::= void idMetVar <ArgsFormales> <Bloque>
<MetodoResto> ::= <MetodoVoid> | <Tipo> idMetVar <ArgsFormales> <Bloque>
```

- **Sentencias de expresión:** <Asignacion> y <Llamada> derivan ambas en <Expresion>, por lo que se unifican en <Sentencia> ::= <Expresion> ;.
- **If con else opcional** (dos producciones con prefijo común):

```
<If> ::= if ( <Expresion> ) <Sentencia>
        | if ( <Expresion> ) <Sentencia> else <Sentencia>
```

se transforma en:

```
<If> ::= if ( <Expresion> ) <Sentencia> <ElseOpcional>
<ElseOpcional> ::= else <Sentencia> | eps
```

- **Expresión con asignación opcional:**

```
<Expresion> ::= <ExpresionCompuesta> <OperadorAsignacion> <ExpresionCompuesta>
                | <ExpresionCompuesta>
```

se transforma en:

```

<Expresion> ::= <ExpresionCompuesta> <ExpresionResto>
<ExpresionResto> ::= <OperadorAsignacion> <ExpresionCompuesta> | eps

```

- **Primarios:** <AccesoVar> y <LlamadaMetodo> comparten el prefijo idMetVar; <CrearArreglo> y <LlamadaConstructor> comparten new y la estructura de <TipoBase> (que a su vez incluye <TipoReferencia>):

```

<Primario> ::= idMetVar <MetodoOVar> | new <ArregloOConstructor> | ...
<MetodoOVar> ::= <ArgsActuales> | eps
<ArregloOConstructor> ::= <TipoReferencia> <AOC2>
                        | <TipoPrimitivo> <DimensionesConTamano>
                        | idGen <DimensionesConTamano>
<AOC2> ::= <ArgsActuales> | <DimensionesConTamano>

```

De la misma forma, en <RResto> los encadenados . idMetVar y . idMetVar <ArgsActuales> se factorizan en . idMetVar <MetodoOVar>.

- **Dimensiones con tamaño y lista de expresiones** (prefijo común):

```

<DimensionesConTamano> ::= [ <Expresion> ] <DCTResto>
<DCTResto> ::= [ <Expresion> ] <DCTResto> | eps
<ListaExps> ::= <Expresion> <ListaExpsResto>
<ListaExpsResto> ::= , <ListaExps> | eps

```

4.3 Observaciones sobre la LL(1)-cidad

- **Else colgante:** es la única construcción residual que, en forma estricta, no es LL(1) (Primeros($\text{else } S$) $\cap$ Siguientes(<ElseOpcional>) contiene else). El esquema recursivo la resuelve de manera determinista consumiendo el else apenas aparece: el else se asocia al if más cercano, como en Java.
- **Doble uso de los tokens op< y op>** (genéricos vs. operadores relacionales): no genera conflictos porque los genéricos sólo aparecen en contextos de tipo (tras idClase) y los siguientes de las reglas de genericidad nunca incluyen op< ni op>. El contexto del parser determina la lectura.
- **Unarios vs. binarios (+ y -):** los binarios los consume <EResto> después de una <ExpresionBasica> completa, y dentro de <ExpresionBasica> los primeros de <OperadorUnario> y de <Operando> son disjuntos.

5 Gramática final utilizada

Versión final LL(1) (equivalente a la original: genera el mismo lenguaje). El símbolo eof corresponde al token EOF (lexema \$). Esta versión incorpora las extensiones de los logros declarados en la sección *Logros*: <Miembro>, <MiembroSinVisibilidad>, <VisibilidadOpcional>, <RestoTipoOConstructor>, <TipoFinalOConstructor>, <MetodoOAtributo>, <MetodoInterfaz>, <ExpresionBasica> y <PosfijoOpcional>. En particular, el constructor con visibilidad implícita se deriva por idClase <RestoTipoOConstructor> (no hay una regla <Constructor> separada); los constructores con visibilidad explícita public/private también entran por esa vía, con la <VisibilidadOpcional> consumida como prefijo del miembro.

```

<Inicial> ::= <ListaClases> eof

<ListaClases> ::= <Clase> <ListaClases> | <Interfaz> <ListaClases> | eps

<Clase> ::= class idClase <GenericidadOpcional> <HerenciaOpcional> { <ListaMiembros> }

```

```

<Interfaz> ::= interface idClase <GenericidadOpcional> <ExtensionOpcional> { <ListaMetodosInterfaz> }

<GenericidadOpcional> ::= < idGen > | eps

<HerenciaOpcional> ::= extends <TipoReferencia> | implements <TipoReferencia> | eps

<ExtensionOpcional> ::= extends <TipoReferencia> | eps

<ListaMiembros> ::= <Miembro> <ListaMiembros> | eps

<ListaMetodosInterfaz> ::= <MetodoInterfaz> <ListaMetodosInterfaz> | eps

<Miembro> ::= <VisibilidadOpcional> <MiembroSinVisibilidad>
<MiembroSinVisibilidad> ::= <MetodoVoid>
<MiembroSinVisibilidad> ::= static <MetodoResto>
<MiembroSinVisibilidad> ::= <TipoPrimitivo> <DimensionesOpcionales> idMetVar <MetodoOAtributo>
<MiembroSinVisibilidad> ::= idGen <DimensionesOpcionales> idMetVar <MetodoOAtributo>
<MiembroSinVisibilidad> ::= idClase <RestoTipoOConstructor>
<VisibilidadOpcional> ::= public | private | eps
<RestoTipoOConstructor> ::= <TipoGenericoOpcional> <TipoFinalOConstructor>
<TipoFinalOConstructor> ::= <DimensionesOpcionales> idMetVar <MetodoOAtributo>
<TipoFinalOConstructor> ::= <ArgsFormales> <Bloque>
<MetodoOAtributo> ::= ;
<MetodoOAtributo> ::= = <ExpresionCompuesta> ;
<MetodoOAtributo> ::= <ArgsFormales> <Bloque>
<MetodoVoid> ::= void idMetVar <ArgsFormales> <Bloque>
<MetodoResto> ::= <MetodoVoid>
<MetodoResto> ::= <Tipo> idMetVar <ArgsFormales> <Bloque>

<MetodoInterfaz> ::= <VisibilidadOpcional> <TipoMetodo> idMetVar <ArgsFormales> ;

<TipoMetodo> ::= <Tipo> | void

<Tipo> ::= <TipoBase> <DimensionesOpcionales>

<TipoBase> ::= <TipoPrimitivo> | <TipoReferencia> | idGen

<DimensionesOpcionales> ::= [] <DimensionesOpcionales> | eps

<TipoReferencia> ::= idClase <TipoGenericoOpcional>

<TipoPrimitivo> ::= boolean | char | int

<TipoGenericoOpcional> ::= < <InstanciadoOParametrico> > | eps

<InstanciadoOParametrico> ::= idGen | idClase

<ArgsFormales> ::= ( <ListaArgsFormalesOpcional> )

<ListaArgsFormalesOpcional> ::= <ListaArgsFormales> | eps

<ListaArgsFormales> ::= <ArgFormal> <LAFResto>
<LAFResto> ::= , <ArgFormal> <LAFResto> | eps

<ArgFormal> ::= <Tipo> idMetVar

<Bloque> ::= { <ListaSentencias> }

<ListaSentencias> ::= <Sentencia> <ListaSentencias> | eps

<Sentencia> ::= ;
<Sentencia> ::= <Expresion> ;

```

```

<Sentencia> ::= <VarLocal> ;
<Sentencia> ::= <Return> ;
<Sentencia> ::= <If>
<Sentencia> ::= <While>
<Sentencia> ::= <Bloque>

<VarLocal> ::= var idMetVar = <ExpresionCompuesta>

<Return> ::= return <ExpresionOpcional>

<ExpresionOpcional> ::= <Expresion> | eps

<If> ::= if ( <Expresion> ) <Sentencia> <ElseOpcional>
<ElseOpcional> ::= else <Sentencia> | eps

<While> ::= while ( <Expresion> ) <Sentencia>

<Expresion> ::= <ExpresionCompuesta> <ExpresionResto>
<ExpresionResto> ::= <OperadorAsignacion> <ExpresionCompuesta> | eps

<OperadorAsignacion> ::= =

<ExpresionCompuesta> ::= <ExpresionBasica> <ECResto>
<ECResto> ::= <OperadorBinario> <ExpresionBasica> <ECResto> | eps

<OperadorBinario> ::= || | && | == | != | < | > | <= | >= | + | - | * | / | %

<ExpresionBasica> ::= <OperadorUnario> <Operando> <PosfijoOpcional>
<ExpresionBasica> ::= <Operando> <PosfijoOpcional>
<PosfijoOpcional> ::= ++ | -- | eps

<OperadorUnario> ::= + | - | !

<Operando> ::= <Primitivo>
<Operando> ::= <Referencia>

<Primitivo> ::= true | false | intLiteral | charLiteral | null

<Referencia> ::= <Primario> <RResto>
<RResto> ::= . idMetVar <MetodoOVar> <RResto>
<RResto> ::= <AccesoArreglo> <RResto>
<RResto> ::= eps

<Primario> ::= this
<Primario> ::= stringLiteral
<Primario> ::= idMetVar <MetodoOVar>
<Primario> ::= new <ArregloOConstructor>
<Primario> ::= <LlamadaMetodoEstatico>
<Primario> ::= <ExpresionParentizada>
<ArregloOConstructor> ::= <TipoReferencia> <AOC2>
<ArregloOConstructor> ::= <TipoPrimitivo> <DimensionesConTamano>
<ArregloOConstructor> ::= idGen <DimensionesConTamano>
<AOC2> ::= <ArgsActuales> | <DimensionesConTamano>
<MetodoOVar> ::= <ArgsActuales> | eps

<ExpresionParentizada> ::= ( <Expresion> )

<LlamadaMetodoEstatico> ::= idClase . idMetVar <ArgsActuales>

<DimensionesConTamano> ::= [ <Expresion> ] <DCTResto>
<DCTResto> ::= [ <Expresion> ] <DCTResto> | eps

<ArgsActuales> ::= ( <ListaExpsOpcional> )

```

```

<ListaExpsOpcional> ::= <ListaExps> | eps

<ListaExps> ::= <Expresion> <ListaExpsResto>
<ListaExpsResto> ::= , <ListaExps> | eps

<AccesoArreglo> ::= [ <Expresion> ]

```

6 Errores sintácticos

Al detectarse un error se lanza `ExcepcionSintactica` con el lexema y la línea del token actual, el módulo principal la captura, reporta el error y finaliza la ejecución (no hay recuperación: modo pánico fuera del alcance de esta entrega). Los errores léxicos se reportan con el mismo formato, ya que la excepción `ExcepcionLexica` se propaga a través del sintáctico. En el código de error el lexema del token EOF es \$.

```

Error Sintactico en linea <N>: <mensaje>
[Error:<lexema>|<N>]

```

El analizador detecta dos tipos de errores sintácticos:

- **Falla de match:** el token actual no coincide con el terminal que exige la producción en curso. Mensaje **se esperaba <esperado> se encontro "<encontrado>"**. Por ejemplo, en `class {:`

```

Error Sintactico en linea 1: se esperaba un id de clase se encontro "{"
[Error:{|1]

```

- **Token fuera de los primeros:** el token actual no pertenece a los primeros de ninguna producción de un NT sin producción vacía. Mensaje **se encontro "<encontrado>" cuando se esperaba <alternativas>**. Por ejemplo, un atributo sin `::`

```

Error Sintactico en linea 4: se encontro "]" cuando se esperaba ',' o los
argumentos formales de un metodo
[Error:}|4]

```

Ejemplos adicionales de reportes:

Fuente (fragmento)	Reporte
<code>a = b+4 hola;</code>	[Error:hola 5] (se esperaba ; tras la expresión)
falta <code>}</code> al final del archivo	[Error:\$ 4] (EOF inesperado)
<code>var x;</code>	[Error:; 4] (se esperaba = tras la variable)
<code>new int [];</code>	[Error:] 4] (dimensión sin expresión de tamaño)
caracter <code>#</code> dentro de un miembro	[Error:# 3] (error léxico propagado)

7 Formato de salida

7.1 Análisis exitoso

```

Compilacion Exitosa
[SinErrores]

```

7.2 Análisis con errores

Mensaje descriptivo de la naturaleza del error indicando el lexema del token con el que se produjo y su número de línea, seguido del código de error (ejemplos en la sección anterior). Toda la salida se realiza con `System.out`.

8 Compilación y ejecución

No se usa ninguna herramienta de build: solo se necesitan `javac`, `jar` y `java` de cualquier JDK 21 o superior (probado con JDK 25).

```
./Compilar.sh          # compila y genera Compilador.jar  (Windows: Compilar.bat)
java -jar Compilador.jar resources/sinErrores/sintCorrecto01.java
./CorrerTests.sh       # compila y ejecuta los testers JUnit  (Windows: CorrerTests.bat)
```

`CorrerTests.sh` descarga automáticamente `junit-4.13.2.jar` y `hamcrest-core-1.3.jar` en `lib/` si no están (requiere `curl`); si no hay red, se pueden descargar a mano de Maven Central y dejarlos en esa carpeta.

8.1 Comandos manuales equivalentes

Compilar (desde `codigo/`):

```
javac -d build/classes \
  src/main/java/moduloprincipal/*.java \
  src/main/java/analizadorlexico/*.java \
  src/main/java/analizadorsintactico/*.java \
  src/main/java/sourcemanager/*.java
jar cfe Compilador.jar moduloprincipal.ModuloPrincipal -C build/classes .
```

Correr los tests (desde `codigo/`, con JUnit 4.13.2 y Hamcrest 1.3 en `lib/`):

```
javac -cp "build/classes:lib/junit-4.13.2.jar:lib/hamcrest-core-1.3.jar" \
  -d build/test-classes src/test/java/test/*.java

java -cp "build/classes:build/test-classes:lib/junit-4.13.2.jar:\
lib/hamcrest-core-1.3.jar" org.junit.runner.JUnitCore \
  test.TesterDeCasosSinErrores test.TesterDeCasosConErrores
```

En Windows el separador del classpath es `;` en lugar de `..`.

Los testers usan el working directory `codigo/` para acceder a `resources/sinErrores/` y `resources/conErrores/` (los scripts ya se posicionan ahí; los comandos manuales deben correrse desde `codigo/`). Los casos sin errores deben ser aceptados (la salida debe contener `[SinErrores]`); los casos con errores llevan en su primera línea el código esperado con el formato `///[Error:lexema|nroLinea]` y el tester verifica que aparezca en la salida.

9 Tests

Se incluyen los casos provistos por la cátedra y casos propios desarrollados para esta etapa (el diseño de los casos de prueba es responsabilidad del alumno). Resultado actual: **29/29 casos OK** (14 sin errores y 15 con errores).

9.1 Casos propios

- `sintCorrecto05.java` - interfaz con genericidad y extensión, varios métodos.
- `sintCorrecto06.java` - clase genérica con herencia genérica, atributos de todos los tipos (primitivo, arreglo, clase genérica, parámetro de tipo) y constructor.
- `sintCorrecto07.java` - `if/else` anidados, `while`, `var`, `return` con y sin expresión, bloques anidados y sentencia vacía.
- `sintCorrecto08.java` - expresiones unarias, binarias y parentizadas con todos los operadores y literales.

- `sintCorrecto09.java` - referencias encadenadas, creación de arreglos con `new`, accesos con índice y `this`.
- `sintCorrecto10.java` - constructores, `new` de clases con y sin genericidad, `new` de parámetro de tipo y llamadas a métodos estáticos.
- `sintCorrecto11.java` - `implements`, tipos arreglo en parámetros y retorno, varios argumentos formales y actuales.
- `sintError05.java` - falta el id de clase (`class {}`).
- `sintError06.java` - EOF inesperado (falta la llave de cierre), reporta lexema `$`.
- `sintError07.java` - falta `;` en un atributo.
- `sintError08.java` - operador binario sin operando derecho.
- `sintError09.java` - método de interfaz con cuerpo (se esperaba `;`).
- `sintError10.java` - `new int[]` sin tamaño de dimensión.
- `sintError11.java` - paréntesis del `if` sin cerrar.
- `sintError12.java` - `var` sin inicializador.
- `sintError13.java` - `extends` sin tipo.
- `sintError14.java` - acceso encadenado sin identificador (`a. = 3;`).
- `sintError15.java` - error léxico detectado durante el análisis sintáctico (propagación de `ExcepcionLexica`).

10 Logros

Se implementan los siguientes logros opcionales de la Etapa 2, como extensiones aditivas de la gramática LL(1) base (cada uno con casos de prueba propios en `resources/sinErrores/`):

- **Atributos Inicializados:** los atributos pueden inicializarse al declararse (`int x = 5;`). Se agrega la producción
`<Metodo0Atributo> ::= = <ExpresionCompuesta> ;`.
- **Operadores Posfijos E2:** operadores unarios posfijos de incremento y decremento (`++` y `--`) tras un operando. Se agrega `<Posfijo0Opcional> ::= ++ | -- | eps` al final de `<ExpresionBasica>`. Los tokens `op++/op--` ya eran reconocidos por el léxico de la Etapa 1.
- **Visibilidad Mejorada E2:** los miembros pueden declararse con visibilidad explícita `public/private` u omitirla (implícita), para atributos, métodos, constructores y métodos de interfaz. La palabra reservada `private` se agrega al léxico. Al re-factorizarse `<Miembro>` para el prefijo opcional de visibilidad, el constructor sin visibilidad explícita se mantiene soportado mediante la factorización `idClase <RestoTipo0Constructor>` (tras `idClase`, los paréntesis de los argumentos formales indican constructor; un tipo genérico opcional, dimensiones o `idMetVar` indican atributo o método tipado).

De la etapa anterior se conserva vigente el logro del **Manejador de Archivos Eficientes** (`SourceManagerEficiente`), que sigue siendo el módulo de lectura del fuente. La multi-detección de errores léxicos de la Etapa 1 no aplica en esta etapa: el módulo principal ahora finaliza al primer error detectado (sintáctico o léxico), según lo exigen las pautas de la Etapa 2.